



Técnico em Informática

Programação para Web

Prof. Robson Pires Borges

PHP – Sanitização / Segurança



Programação Para Web – Prof. Robson.



Vulnerabilidades

- SQL Injection - XSS - Cross Site Scripting
- PHP Injection
- Spoofing de formulários
- Mudar o nome da sessão
- Forma de validação dos dados
- Usar Captcha para evitar ataques de força bruta
- Sempre criptografar a senhas

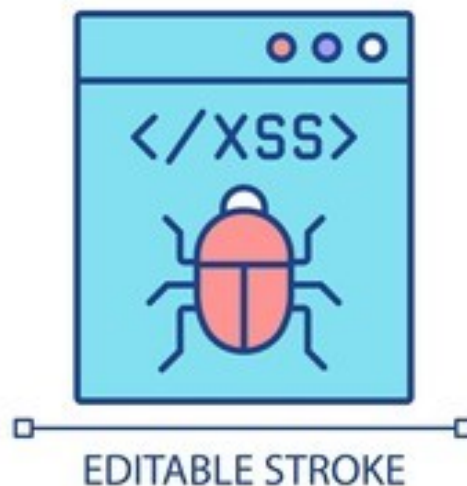


O que é sanitização?

Definição:

Processo de limpar e validar dados de entrada para evitar:

- Injeção de SQL
- XSS (Cross-Site Scri
- Dados corrompidos





O que é sanitização?

Exemplo:



```
// Dados não sanitizados:  
$user_input = "<script>alert('Hack!');</script>";  
  
// Dados sanitizados:  
$clean_input = htmlspecialchars($user_input);  
</body>  
</html>
```



Por que sanitizar?

Riscos sem sanitização:

1. **Segurança:** Ataques como SQL Injection podem roubar dados.
2. **Integridade:** Dados inválidos quebram a aplicação.
3. **Confiabilidade:** Usuários podem inserir scripts maliciosos.



```
-- Se o usuário digitar: ' OR '1'='1  
-- Consulta vulnerável:  
SELECT * FROM usuarios WHERE login = '$user' AND senha = '$pass';
```



Funções de Sanitização no PHP

Principais funções:	Uso
<code>htmlspecialchars()</code>	Converte caracteres HTML em entidades (evita XSS)
<code>filter_var()</code>	Filtra dados com regras pré-definidas (email, URL, etc.)
<code>strip_tags()</code>	Remove tags HTML/PHP
<code>mysqli_real_escape_string()</code>	Escapa strings para SQL (útil para MySQL)



```
$email = "user@example.com<script>";  
$clean_email = filter_var($email, FILTER_SANITIZE_EMAIL);
```

Exemplo Prático (XSS)

- **Problema:** Um formulário que exibe dados não sanitizados.



```
// Formulário perigoso:
```

```
echo "Comentário: " . $_POST['comentario'];
```

```
// Solução (sanitizado):
```

```
echo "Comentário: " . htmlspecialchars($_POST['comentario']);
```

Resultado:

- Sem sanitização: `<script>alert('Ataque!');</script>` → Executa o script.
- Com sanitização: Exibe o texto literalmente.

Exemplo Prático (SQL Injection)

- Consulta vulnerável:



```
$user = $_POST['user'];  
$query = "SELECT * FROM usuarios WHERE user = '$user'";
```

- Solução:



```
$user = mysqli_real_escape_string($conn, $_POST['user']);  
// OU (melhor ainda): usar Prepared Statements!
```



- Sempre valide e sanitize entradas.
- Use `filter_var()` para emails, URLs, etc.
- Prefira *Prepared Statements* (PDO/MySQLi) para SQL.
- Para saída HTML, use `htmlspecialchars()`.

Regra de ouro:

"Nunca confie em dados do usuário!"



Desafio:

- Crie um formulário HTML com campos (nome, email, comentário).
- Sanitize os dados no PHP usando htmlspecialchars() e filter_var().
- Exiba os dados sanitizados na página.



```
$nome = htmlspecialchars($_POST['nome']);  
$email = filter_var($_POST['email'], FILTER_SANITIZE_EMAIL);
```



Referências

- G553p Glazar, Jean Eduardo, Programação para web : Curso técnico em informática / Jean Eduardo Glazar. – Colatina: IFES, 2011.102 p. : il. ISBN: 978-85-62934-37-7
- PHP (Linguagem de programação de computador). 2. Banco de dados. 3. Sites da Web – Desenvolvimento. I. Glazar, Jean Eduardo. II. Instituto Federal do Espírito Santo. III. Título.
- <https://qastack.com.br/programming/3126072/what-are-the-best-php-input-sanitizing-functions>
- <http://phpbrasil.com/artigo/cA6agoMO2IP/principais-falhas-de-seguranca-em-php>
- http://php.net/manual/pt_BR/mysqli.error.php
- <https://www.youtube.com/watch?v=AUPYmk4Coc4>
- <https://www.youtube.com/watch?v=hNxyFBcCsQo>
- Screenshots from <https://carbon.now.sh/>
- Manual do PHP: Sanitização
- OWASP - Boas práticas de segurança

